

# Gjennomgang av Arbeidskrav 1 - Algoritmer

Helhetsinntrykket er godt: alle seks områder er dekket, Git-historikken har 54 meningsfulle commits, README-filene per oppgave er grundige, og oppgave 4 og 5 har solid testdekning. Under er kun det jeg mener faktisk bør rettes før innlevering, sortert per oppgave og med de tyngste først innenfor hver. Rene stilspørsmål er utelatt. **FEIL** = koden gjør noe galt **KRAV** = oppgaveteksten ber om noe som mangler **FAGLIG** = forklaring som ikke stemmer.

## Oppgave 1 - Lineært søk og binærsøk

### 1. Binærsøket velger halvdel ut fra feil sammenligning **FEIL**

```
if (array[left] <= mid)    // sammenligner en VERDI med en INDEKS
```

`array[left]` er et tall som ligger *i* arrayet, mens `mid` er en *posisjon*. Det er to helt forskjellige størrelser, og å sammenligne dem gir ikke mening. Valget av halvdel må avgjøres ved å sammenligne *mid* med verdien du leter etter.

Konsekvensen på testarrayet oppgaven krever, `[2, 5, 7, 9, 11, 13, 17]`:

|           |    |    |    |   |    |    |    |
|-----------|----|----|----|---|----|----|----|
| Søk etter | 2  | 5  | 7  | 9 | 11 | 13 | 17 |
| Koden gir | -1 | -1 | -1 | 3 | 4  | 5  | -1 |
| Riktig    | 0  | 1  | 2  | 3 | 4  | 5  | 6  |

Fire av sju verdier meldes som «finnes ikke». Testene dine treffer akkurat de to tilfellene som virker (13 og 4), så alt ser grønt ut. Dette er den viktigste enkeltfeilen i besvarelsen.

```
if (array[mid] < target) left = mid + 1;
else                    right = mid - 1;
```

### 2. array[mid - 1] krasjer når mid er 0 **FEIL**

Når `mid` er 0 blir `array[mid - 1]` til `array[-1]` og du får `IndexOutOfRangeException`. Det skjer f.eks. med to elementer der målet ligger på indeks 0. Merk at KI-svaret du selv har dokumentert i `Readme1.md` påpeker nøyaktig dette («out of bounds når `mid == 0`»), men fiksen ble bare lagt inn i den *andre* if-blokken. Den første står fortsatt uten sjekk.

```
if (array[mid] == target && (mid == 0 || array[mid - 1] != target))
```

### 3. Forsøket med usorterte data mangler **KRAV**

Oppgaven ber eksplisitt om: kjør binærsøket på en usortert kopi, observer resultatet og forklar hvorfor det ikke er pålitelig. `Readme1` forklarer at sortering kreves, men selve forsøket er ikke gjort noe sted. Testen `BinarySearch_SameNumbers_ShouldReturnFirstIndex` bruker faktisk et usortert array (`{2,5,4,4,4,3}`), men slår fast at svaret blir *riktig* – altså motsatt av poenget. Bruk et sortert array med duplikater der (f.eks. `{2,4,4,4,5}`), og lag et eget lite forsøk som viser at usortert input gir tilfeldig resultat.

**Mindre, men verdt et minutt:** telleren skrives ut per runde, men summeres aldri til ett tydelig totaltall per søk, og siden den ikke returneres kan ingen test sjekke den. Begge metodene kaster også unntak for array med ett element – et enkeltelement er en helt gyldig søkeinput, og oppgaven ber deg teste nettopp det tilfellet.

## Oppgave 2 - Egen generisk datastruktur

### 4. Ingen test viser at køen faktisk er FIFO **KRAV**

Alle tre testene sjekker `Size`, altså kapasiteten til det interne arrayet. Ingen av dem sjekker at `Dequeue` leverer elementene i samme rekkefølge som de ble lagt inn – som er hele poenget med en kø. Siden `Enqueue` skyver alt oppover og `Dequeue` leser fra motsatt ende, ville en forskyvning på én indeks gått rett gjennom testene. Legg inn: `enqueue 1, 2, 3` og assert at `dequeue` gir 1, så 2, så 3.

### 5. Tom kø håndteres ikke konsekvent **KRAV**

Oppgaven ber om «tydelig og konsekvent» håndtering. I dag kaster `Dequeue` unntak, mens `Peek` bare gjør ingenting i stillhet. `Peek` er dessuten `void` og skriver rett til konsollen, så den kan verken testes eller brukes videre. Verre: menyvalg 6 leser `queue. array[queue.IndexFront]`, og på tom kø er `IndexFront` lik -1, så programmet krasjer ut av menyen. Velg én linje – la begge kaste, eller la begge returnere noe – og gi `Peek` en returverdi.

## Oppgave 3 - QuickSort

### 6. Sammenligninger telles ikke **KRAV**

Oppgaven krever at du teller *sammenligninger* og én annen relevant operasjon. Du teller bytter og splitter – det dekker «den andre operasjonen» – men selve sammenligningen i `Partition` telles aldri:

```
for (int j = low; j < pivotIndex; j++)
{
    comparisonCounter++;           // <- denne mangler
    if (array[j] < array[pivotIndex]) { ... }
```

Tellingen kjøres dessuten bare på `{3, 5, 8, 5}`, og ingen tall står i `Readme3.md`. Kjør de samme tellerne på det tilfeldige settet og på `[1,2,3,4,5]`, og skriv tallene inn i README – det er akkurat dén sammenligningen som viser hvorfor sortert input gir verstetilfellet  $O(n^2)$ , og den er gull i videoen.

### 7. Readme3 kaller det rekursive steget for basissteget **FAGLIG**

`Readme3` sier: «hvis `low` er mindre enn `high` så kjøres rekrusjonen. Det er dette som er basissteget.» Det er motsatt. Når `low < high` er sant, er det det *rekursive* steget som kjører. Basissteget er tilfellet der `low >= high`, altså at delområdet har null eller ett element – da skjer ingenting, metoden når slutten og returnerer. Det er dette som stopper rekursjonen. Samme forveksling går igjen i setningen om at pivot er riktig plassert «etter at basissteget er kjørt» (det skal være *partisjoneringen*).

## Oppgave 4 - Graf og BFS

### 8. Ingen test for det fasitsvaret oppgaven oppgir **KRAV**

Oppgaveteksten sier rett ut at Majorstuen → Grønland skal gi 4. Det tallet finnes ingen steder som en assert – kun som en utskrift i konsollen. Samme for Ullevål stadion → Tøyen. To korte tester er billig forsikring, og det er nettopp det sensor vil lete etter.

### 9. Returverdien ved «ingen rute» er ikke dokumentert i README **KRAV**

Oppgaven ber deg «definere og dokumentere tydelig» hva som returneres når det ikke finnes rute. I dag står 0 kun som en kodekommentar. Skriv det inn i `Readme4`. Vurder samtidig å bytte til -1: 0 betyr naturlig «null stopp unna», altså samme stasjon, så verdien er tvetydig. Du har riktignok lukket det hullet ved å kaste unntak når start og mål er like, men -1 sier «finnes ikke» utvetydig og er konsistent med `LinearSearch` i oppgave 1.

## Oppgave 5 - DFS

### 10. Manglende startstasjon krasjer begge DFS-variantene **FEIL**

Både `TraverseRecursiveDFS` og `TraverseIterativeDFS` henter naboer med `graph[node]`. Finnes ikke startnoden i grafen, får du `KeyNotFoundException` og programmet stopper – altså akkurat det oppgaven ber deg unngå («uten at programmet avsluttes ukontrollert»). Du har allerede løsningen i klassen: `Neighbors(node)` bruker `TryGetValue` og returnerer tom liste. De to traverseringene er de eneste stedene som ikke bruker den. KI-dialogen du har dokumentert i `Readme4` beskriver nøyaktig denne fiksen for BFS.

`TraverseBFS` krasjer ikke, men returnerer en liste som inneholder startnoden selv om den ikke finnes i grafen – den melder altså at den har besøkt en stasjon som ikke eksisterer. En sjekk øverst i begge klassene løser begge deler.

### 11. Demoen i Program.cs tester feil stasjon og påstår noe usant **FEIL**

```
var isPath = DFS.TryFindPath("Majorstuen", "Tøyen");
Console.WriteLine("There is noe path between Majorstuen and Tøyen. Ispath = " + isPath);
```

«Tøyen» er en skrivefeil, så metoden returnerer `false` utelukkende fordi stasjonen ikke finnes. Det *finnes* en rute mellom Majorstuen og Tøyen (8 stopp), så utskriften påstår det motsatte av sannheten. Oppgaven ber deg vise at det ikke finnes rute til den *isolerte* stasjonen, så kallet bør være `TryFindPath("Majorstuen", "NewNode")` etter at noden er lagt til. Testen din `TryFindPath_MajorstuenToIsolatedNode_ShouldReturnFalse` gjør dette helt riktig – det er bare demoen som er feil. Rett også «noe» → «no». Jeg ville fikset denne først, siden konsollen er det du skal vise i videoen.

### 12. Readme5 kaller foreach-løkken for basissteget **FAGLIG**

`Readme5` sier: «Basissteget er `foreach` løkken som leter gjennom naboene samt sjekken om naboen er besøkt.» `foreach`-løkken og `if`-en er det *rekursive* steget – det er de som produserer nye kall. Basissteget er *situasjonen* der ingen nabo slipper gjennom `if`-en, slik at løkka aldri kjører kroppen sin og metoden når slutten. Du skriver faktisk dette helt korrekt noen linjer lenger ned («I koden finnes ingen return for basissteget. Men når alle naboer er besøkt...»), så README-en motsier seg selv. Stryk eller omformuler den første setningen.

Småplukk i samme avsnitt: «LIFO (last in last out)» skal være *last in, first out*.

## Felles og innlevering

**13. Det mangler en samlet README.md** KRAV

Kravet er at README.md skal beskrive navn, hvordan løsningen bygges og kjøres, testoversikt, resultater, begrensninger og KI-bruk. De fem README-filene per oppgave dekker algoritmiske valg godt, men ingen steder står navnet ditt, hvordan man faktisk kjører prosjektet (åpne `KodeAlgo.slnx` i Visual Studio, eller `dotnet run --project KodeAlgo` og `dotnet test`), eller en oppsummering av resultatene. Filen `KodeAlgo/KodeAlgo/Readme.md` inneholder bare KI-dokumentasjonen. Legg en README.md øverst i ZIP-en som binder det sammen og lenker til de fem andre – og legg videolenken der.

**14. Byggemapper ligger i ZIP-en** KRAV

`bin/`, `obj/` og `.vs/` utgjør omtrent 7,7 MB, og oppgaven ber deg uttrykkelig la være å pakke med mapper som lett kan genereres på nytt. `.gitignore`-en din er riktig satt opp – ingenting av dette er sporet i Git – så det er selve zippingen som tar dem med. Slett mappene før du pakker.